ChatGPT

Stable Matching Design

You

I'm going to build a mentoring system as a project for class. In this project, I will be using AI for various things. I will be completely in charge of the prompts. this means you cannot generate, rewrite, or improve my prompts. AI may handle the algorithm, implementation details, and coding, I must understand everything in this project.

Big picture: it's a mentoring system. a system that assigns each new recruit a mentor who is a more experience employee.

subsystems:

one subsystem, and what I intuitively think will be the most important is the ranking system.
Mentors rank their preferences amongst mentees. Mentees rank their preferences amongst mentors.

After preferences are in the system:
a system administrator should have the privilege of pressing a button to assign mentors to all mentees.

My thoughts, questions, conceptual design, ideas for big system, subsystems, UI/UX, privileges and restrictions, and how everything interacts with each other is on paper. I will finish that up, upload it. I want to build the backend in C++.

I'll begin with the core system, the ranking system. The constraints are given. m mentors and n mentees, m >= n and must be 1:1 assignemnt. There may not be a mentee X & mentor Y such that x prefers y to their assigned member & Y prefers X to their assigned mentee.

A mentor assignment is !reasonable! if the above situation does not

Take your curiosity further
Scan to get the ChatGPT app

occur. reasonability can be thought of as a true or false. it either is or isnt. if the condtions above are true, then reasonability is true, so the assignments are valid and system administrators request to assign mentors to mentees is accepted. that's what button does. so if a mentor changes his preference for a mentee, and that mentee isnt the one he has, the time after has reasonability = false and the time before the change reasonability = true. so any assigned mentor or mentee change in preference will cause reasonability = false.

don't build anything out yet. this is just my initial chat

ChatGPT

Understood. **You own all prompts. I won't generate, rewrite, or improve them.** I can help with algorithms, implementation decisions, and C++ code, with explanations so you understand the system. We won't build anything yet.

Your core subsystem is a **stable matching problem**. Your term *reasonable* corresponds to what algorithms call *stable*.

With m mentors and n mentees, where $m \geq n$, your assignment constraints mean:

- Every mentee receives exactly one mentor.
- Each mentor receives at most one mentee.
- Exactly $m - n$ mentors remain unassigned.

The situation you want to prevent is called a **blocking pair**: a mentee X and mentor Y, who are not assigned to each other, would both prefer being assigned to each other over their current situations. For an unassigned mentor, we need to specify whether they prefer any acceptable mentee over remaining unassigned.

Your Boolean interpretation makes sense:

$$\text{reasonable}(A, P) = \begin{cases} \text{true}, & \text{if assignment } A \text{ has no blocking pair under preferences } P, \\ \text{false}, & \text{otherwise.} \end{cases}$$

Take your curiosity further
Scan to get the ChatGPT app

There is **one important correction**: a preference change does **not necessarily** make an existing assignment unreasonable. It only does so if the change creates a blocking pair.

For example, suppose:

- Mentor Y is assigned to mentee A.
- Mentee X is assigned to mentor Z.
- Y changes their preferences and now prefers X over A.

If X still prefers Z over Y, there is no blocking pair between X and Y. If X prefers Y over Z, then there is a blocking pair, and reasonability becomes false.

So **a preference change can trigger a new stability check; it does not predetermine the result.**

Also, conceptually, the administrator's button requests the system to **compute assignments**. Checking whether those assignments satisfy your constraints and are reasonable is a related but distinct responsibility. That distinction will help when you organize the subsystems later.

This gives us a clear starting point. We can leave the remaining rules and design decisions open until you upload your paper notes.

You

I've designed some rough idea of the stages of development. I've determined that I want to use the gale-shapley (deferred acceptance) algorithm. This will be the matching algorithm. This will be Phase 1.

There's also a need to verify stability. Stability verification will independetly verify that assignments are reasonable and stable. This will be Phase 2.

Phase 3 is the testing stage. These three stages can be grouped together I think, but give me feedback on that. It's the foundation or core of the project. I think testing test cases, edge cases and algorithmic performance in this phase is best.

So phase 1-3 is the core matching system. I don't want to move onto the rest of the phases until this is established.

Take your curiosity further
Scan to get the ChatGPT app

So you have the scope of the project, I will also mention the rest of the phases I have in mind but things can be subjec to change.

I'm still on the fence whether to use supabase or sqlite. I could use some feed back here., but that will be phase 4, the database design. How users, preferences and the assignments are stored and managed. Consider Phase 5 here as well, which is the API (communication between components).

Phase 6 is the UI. I want a minimalistic approach, I will give some design ideas here. I'm thinking about using lovable, but we'll test a UI generated by GPT 6.1 Sol first.

Stage 7 is integration. conencting the backend, database, and the frontend.

Stage 8-9 is full-system tesitng and deployment.

again,Right now we're focusing on the algorithmic foundation. I will upload some notes I have thus far, but first give me feedback from this prompt and any questions you have

Worked for 12s

ChatGPT

Yes—Phases 1–3 form a coherent "core matching system" milestone. Establishing that foundation before adding storage, an API, or a UI is a sensible boundary. It lets you understand and demonstrate the central algorithm on its own.

I'm treating your message as design discussion. Your prompts remain yours, and we won't build anything yet.

Your first three phases have distinct responsibilities:

Take your curiosity further
Scan to get the ChatGPT app

Phase	Responsibility	Evidence of completion
1. Matching	Use Gale–Shapley to produce assignments from preferences.	You can trace the proposals, rejections, and tentative assignments, and explain termination and stability.
2. Verification	Independently examine the input preferences and resulting assignments.	The verifier detects invalid assignments and identifies a blocking pair when one exists.
3. Testing	Challenge both the matcher and the verifier.	Known examples, edge cases, deliberately incorrect assignments, and performance checks behave as expected.

Keep these phases as milestones, but **test as you develop Phases 1 and 2**, then use Phase 3 for broader validation.

For Phase 2, distinguish two checks:

- **Assignment validity:** every mentee has exactly one mentor, no mentor has multiple mentees, and all assigned participants exist and are eligible.
- **Stability:** no unassigned pair would mutually prefer each other over their current situations.

A matching can satisfy the one-to-one rules and still be unstable. Your verifier should explain the failure—for example, by returning the blocking pair—alongside its Boolean result.

“Independent” should mean the verifier examines the final assignments directly. It should not simply rerun Gale–Shapley and require an identical result: **multiple different stable matchings can exist.**

There are two algorithm decisions to settle early:

- **Who proposes?** With strict preferences, deferred acceptance favors the proposing side: each proposer receives their best attainable partner among stable matchings. That does not mean it maximizes everyone’s combined satisfaction.
- **What constitutes a preference list?** Complete rankings, incomplete rankings, and ties require different assumptions. With complete, strict rankings and every pair acceptable, $m \geq n$ allows Gale–Shapley to match every mentee. With unacceptable or omitted partners, that guarantee can disappear.

Take your curiosity further
Scan to get the ChatGPT app

Your Phase 3 should test the verifier itself using intentionally invalid and unstable assignments. Otherwise, a faulty matcher and faulty verifier could agree and create false confidence. For performance, count proposals as well as elapsed time; with no repeated proposals, there are at most nm proposals.

For the later database decision, **you can defer the choice until the core works**. Keep the matching and verification functions operating on ordinary C++ data structures.

Option	What it provides	My assessment for this project
SQLite	An embedded SQL database without a separate database server.	A strong choice for keeping the C++ application and persistence simple.
Supabase	PostgreSQL plus services including authentication and generated data APIs.	Attractive if hosted accounts and frontend integration become major priorities.

Those platform differences are documented by SQLite and Supabase. www2.sqlite.org +3 My provisional preference is **SQLite for a contained class project**, subject to your eventual login and deployment requirements. Supabase would still leave a role for your C++ API to run matching and enforce administrator privileges.

Your remaining sequence is reasonable. Design Phases 4–5 together, and agree on the API's inputs and outputs before building much of Phase 6. Integration and full-system testing will still need their own milestones.

The questions that matter most before Phase 1 are:

1. Must everyone rank **every person** on the opposite side?
2. Are ties permitted, or must every rank be unique?
3. Can someone mark a potential partner unacceptable?
4. Should mentees or mentors propose—or is that still undecided?

Those answers establish the precise problem your core must solve.

You

It will be mentors that propose since if a mentor doesn't have someone assigned, they don't have any preference at all, so their proposal to any mentee leftover will be accepted.

Take your curiosity further
Scan to get the ChatGPT app

answers to phase 1:

1. Yes, everyone must rank every person on the opposite side,
2. ties are not permitted, every rank must be unique.
3. I don't think this helps, but instead we can do ratings where an assigned mentor and mentee can give ratings that only the server admin can see to the mentee or mentor they wre respectively assigned.
4. mentors should propose

I think it would be cool to make this a bit quirky too. Say a mentor changes his preference list from one he had previously before, he has to put some kind of note on why those changes occured.

we also do need some way to add and remove mentors and mentees. Maybe after a certain time has passed, a mentee can become amentor.

ChatGPT

That settles the main Phase 1 assumptions: mentors propose, everyone ranks every person on the opposite side, and rankings are strict—no ties.

One correction to your reasoning: **an unassigned mentor still has preferences.** Being unassigned means they are available to propose; they propose to the highest-ranked mentee they have not already approached.

A mentee's response depends on their situation:

- **Unassigned:** tentatively accepts the proposal.
- **Already tentatively assigned:** keeps whichever mentor they prefer and rejects the other.

Take your curiosity further
Scan to get the ChatGPT app

So a proposal isn't automatically accepted because the mentor is unassigned. Under your complete rankings and $m \geq n$, the algorithm finishes with every mentee assigned and exactly $m - n$ mentors unassigned. Those remaining mentors still have preference lists.

Choosing mentors as proposers is valid. It means the result is **mentor-optimal among stable matchings**, under these assumptions.

Your answer to question 3 sounds like **every possible pairing is acceptable**, although some are preferred over others. That is consistent with requiring everyone to rank everyone. Ratings can then serve a separate purpose:

Feature	Purpose
Preference rankings	Express desired partners before matching; determine assignments.
Assignment ratings	Provide feedback about an actual mentoring relationship; visible only to administrators.
Preference-change note	Records the participant's explanation for changing their rankings.

The required note is a useful accountability feature. We can preserve it as a later requirement. A ranking change would make the previous stability check **outdated**; a new check determines whether the current assignments remain stable. The note itself doesn't affect the algorithm.

Adding, removing, and promoting participants also belongs in the larger system design. These actions affect the matching inputs:

- **Adding someone:** existing participants need to rank the newcomer, and the newcomer must rank everyone on the opposite side.
- **Removing someone:** their entries must be removed from rankings; removing an assigned mentor leaves a mentee needing reassignment.
- **Promoting a mentee:** changes their role and requires a new preference list over mentees.

Any such change also requires checking that $m \geq n$ still holds.

For now, these are later requirements, not additions to Phase 1. The core can work with a fixed set of participants and preferences for each run. We'll later decide when changes become effective and whether they trigger verification, rematching, or wait until the next assignment round.



Take your curiosity further

Scan to get the ChatGPT app



You

okay, you begin phase 1, lets setup the directories. I will use VS code then initialize it on git and publish it to my github- show me the setup and ill make corrections

Worked for 46s

You

okay, before you begin phase 1, lets setup the directories. I will use VS code then initialize it on git and publish it to my github- show me the setup and ill make corrections

Worked for 6s

ChatGPT

We'll set up the directories first, before implementing Phase 1. I prepared a scaffold you can review:

Download mentor-matching-system-phase1-setup.zip

The root folder is `mentor_matching_system/` :

Path	Purpose
`algorithm/include/`	C++ headers declaring the matching and verification functions.
`algorithm/src/`	Matching implementation in Phase 1; independent verifier in Phase 2.
`tests/`	Tests for matching, verification, edge cases, and performance.
`backend/src/`	Later: C++ API and application logic.

Take your curiosity further
Scan to get the ChatGPT app

Path	Purpose
`database/migrations/`	Later: database schema changes.
`frontend/`	Later: minimal UI.
`docs/initial-design/`	Your original handwritten design and notes.
`docs/ai-chat-logs/`	Your exported project conversations, preserving your prompts.
`docs/screenshots/`	Screenshots for documentation and submission.
`docs/development-log.md`	Record of actual work, checks, and fixes.
`docs/roadmap.md`	Development phases and completion status.
`README.md`	Project overview and agreed matching rules.
`CMakeLists.txt`	C++17 build configuration; no executable targets yet.
`.gitignore`	Excludes generated builds, credentials, and local database files from Git.

The future folders are placeholders. **No matching code has been implemented.**

To inspect it in VS Code:

1. Download and extract the ZIP.
2. Select **File → Open Folder**.
3. Open the extracted **`mentor-matching-system`** folder itself.
4. Review the layout and tell me your corrections.

After the layout is settled, we'll initialize Git and publish it to your GitHub.

You

Okay, I've completed my layout. begin phase 1

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Phase 1 starts with the mentor-proposing Gale–Shapley implementation in C++. We'll build it in small pieces so you can explain every data structure and decision.

Send me your **GitHub repository link**, or upload your completed project folder as a ZIP, so I can follow your actual layout.

The first implementation will use these structures:

Structure	Purpose
Mentor preference lists	Each mentor's mentees, ordered from most to least preferred.
Mentee rank lookup	Quickly compare two mentors from a mentee's perspective.
Current assignments	Track each mentor's and mentee's tentative partner.
Next-proposal position	Remember whom each mentor should approach next.
Queue of available mentors	Track unassigned mentors who still have someone to approach.

The algorithm's central step will be:

1. Take an available mentor.
2. Propose to their next preferred mentee.
3. If the mentee is unassigned, tentatively pair them.
4. Otherwise, the mentee keeps their preferred mentor.
5. The rejected mentor continues proposing if their list isn't exhausted.

An important stopping detail: because m can exceed n, we stop when no unassigned mentor has an untried mentee—not when every mentor is assigned.

We'll begin with the input/output representation and a small traceable example, then implement the proposal loop. Phase 1 will include basic input validation and example checks; the independent stability verifier remains Phase 2.

Take your curiosity further
Scan to get the ChatGPT app

You

https://github.com/ninonyc/mentor-matching-system

Worked for 18s

ChatGPT

I couldn't access the repository: cloning requested authentication, so I haven't inspected or changed your files.

Download it from GitHub using **Code → Download ZIP**, then attach the ZIP here. That will let me implement Phase 1 in your exact layout and explain the code as we go.

You

mentor-matching...
File

Worked for 2m 44s

ChatGPT

Phase 1's matching core is implemented and ready for your review.

Download mentor-matching-system-main.zip

File	Purpose
`algorithm/include/matching.hpp`	Defines preferences, results, and the matching function.
`algorithm/src/matching.cpp`	Input validation and mentor-proposing Gale–Shapley.
`algorithm/src/demo.cpp`	Runnable example with four mentors and three mentees.
`tests/matching_smoke_tests.cpp`	12 initial checks—all passed.

Take your curiosity further
Scan to get the ChatGPT app

File	Purpose
`docs/phase1-walkthrough.md`	Explains the structures, proposal sequence, correctness, and complexity.

Extract it separately, then copy the contents into your existing repository folder, preserving your `.git` directory.

In the VS Code terminal, from your project root:

```
mkdir -p build
c++ -std=c++17 -Wall -Wextra -Wpedantic -I algorithm/include
algorithm/src/matching.cpp algorithm/src/demo.cpp -o build/matching_demo
./build/matching_demo
```

Expected output:

```
Mentor 0 -> Mentee 1
Mentor 1 -> Mentee 0
Mentor 2 -> Mentee 2
Mentor 3 -> Unassigned
Proposals: 9
```

Start by reading `matching.hpp`, then run the example. The walkthrough explains all nine proposals before you examine the loop.

Verified with a Linux C++17 compiler; your Mac build remains to be checked. CMake configuration is included but wasn't executed here. Phase 2 remains pending, and nothing has been pushed to GitHub.

You

Okay this check is verified. The expected output was the output- I think this should be logged as a test

Worked for 33s

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Yes—this counts as a **manual functional test**. I logged it as **P1-M01: PASS**, recording your confirmation that the assignments and nine-proposal count matched the expected output.

Copy the updated development-log.md into your repository's `docs/` folder.

This verifies the example on your machine; broader correctness and stability checks remain pending.

development-log.md
Document

You



Great - I am getting some errors though. I will screenshot them and upload. any screenshots that are relevant, we will document, so I'll add them to the screenshots folder after we evaluate them

Worked for 10s

ChatGPT

These look like IntelliSense configuration errors, rather than a failed build. Your successful demo run is still a valid test.

The likely starting point is `std::optional` in `matching.hpp`. It requires **C++17**. If IntelliSense uses an older standard, it cannot understand those assignment fields and then flags their uses throughout `matching.cpp` and `demo.cpp`.

To check:

1. Press **Cmd + Shift + P**.
2. Select **C/C++: Edit Configurations (UI)**.
3. Find **C++ standard** and select **c++17**.

Take your curiosity further
Scan to get the ChatGPT app

4. Reopen `matching.hpp` and let IntelliSense refresh.

That setting controls the editor's analysis; your terminal command already selects C++17 with `-std=c++17 .` code.visualstudio... +1

If the underlines remain, **hover over `std::optional` and send the exact diagnostic text**. The screenshots show where errors appear, but not their messages, so this diagnosis is provisional.

For documentation, keep the **first screenshot showing `matching.hpp`** as the "before" image. After we confirm the fix, capture the same section without errors. Those two should be enough to document the issue and resolution.

You

done.

ChatGPT

Did the red underlines disappear after selecting **C++17**?

If they did, capture `matching.hpp` without errors for the "after" screenshot. We can then log the IntelliSense configuration issue as resolved. Your earlier successful demo test remains valid.

You

Yes, it has been resolved

Worked for 28s

ChatGPT

Logged **P1-I01: Resolved**—setting IntelliSense to C++17 cleared the editor errors; no algorithm changes were needed.

Replace your local `docs/development-log.md` with the updated development-log.md .

Add the before-and-after screenshots to `docs/screenshots/` . Next, we'll walk through `matching.hpp` so you understand the input and result structures before Phase 2.

Take your curiosity further
Scan to get the ChatGPT app

development-log.md
Document

You

development-log has been updated and screenshots have been uploaded

ChatGPT

The demo test and IntelliSense fix are now documented.

Let's begin the code walkthrough. Open **`algorithm/include/matching.hpp`**. This header defines the matching function's inputs and output; the algorithm itself lives in `matching.cpp`.

Start with:

```
using PreferenceLists = std::vector<std::vector<std::size_t>>;
```

`using` creates a **type alias**: `PreferenceLists` is a shorter name for a vector of vectors containing participant IDs.

For example:

```
PreferenceLists mentors = {
    {2, 0, 1}, // Mentor 0's preferences
    {1, 2, 0}  // Mentor 1's preferences
};
```

The **outer index identifies the mentor**. Each row lists mentee IDs from most to least preferred.

Therefore:

```
mentors[0][0] // 2: Mentor 0's first choice is Mentee 2
mentors[0][1] // 0: Mentor 0's second choice is Mentee 0
```

The stored number is a **person's ID**; its position in the row determines their rank.

Before we move on: what would `mentors[1][2]` return, and what does it mean?

Take your curiosity further
Scan to get the ChatGPT app

You

Mentor 1s 3rds preference- which is 0

ChatGPT

Correct—`mentors[1][2]` returns **0**, meaning **Mentor 1's third choice is Mentee 0**.

Next, consider:

```
std::optional<std::size_t>
```

It holds either a participant ID or **no value**, which represents being unassigned:

```
std::optional<std::size_t> partner; // Unassigned
partner = 0;                       // Assigned to participant 0
partner.reset();                    // Unassigned again
```

We need that distinction because **ID 0 is a valid person**, so it cannot also mean "unassigned."

In our result structure:

```
std::vector<std::optional<std::size_t>> mentor_to_mentee;
```

Each index identifies a mentor; its value identifies their assigned mentee, if any.

What would `mentor_to_mentee[2] = 0;` mean?

You

Let's not do this intermediate questioning portion- Bundle up questions like these for me at the end. I'm going through the code as we build it

ChatGPT

Understood. I'll explain the code as we build, without stopping for intermediate questions. I'll collect review questions at the end of each phase so you can work through them together.

Take your curiosity further
Scan to get the ChatGPT app

You

Begin Phase 2

Worked for 2m 34s

ChatGPT

Phase 2 is implemented and ready for your review. The verifier checks assignments directly, without rerunning Gale–Shapley.

Download mentor-matching-system-phase2-update.zip

Extract the ZIP and copy its contents into your existing project root, preserving the folder paths. Replace `CMakeLists.txt`. Your existing development log and screenshots are excluded from this update.

The verifier returns one of four results:

Result	Meaning
`InvalidPreferences`	Rankings or participant counts violate the input rules.
`InvalidAssignment`	Assignments have missing partners, invalid IDs, or inconsistent records.
`Unstable`	A blocking pair exists; its mentor and mentee IDs are returned.
`Stable`	The assignment is valid and has no blocking pair.

`isReasonable()` returns `true` only for `Stable`.

From your project root, run:

```
mkdir -p build
c++ -std=c++17 -Wall -Wextra -Wpedantic -I algorithm/include
algorithm/src/matching.cpp algorithm/src/stability.cpp
algorithm/src/stability_demo.cpp -o build/stability_demo
./build/stability_demo
```

Take your curiosity further
Scan to get the ChatGPT app

Expected output:

```
Computed assignment reasonable: true
Valid one-to-one assignment; no blocking pair exists
Manual assignment reasonable: false
Blocking pair: Mentor 0 and Mentee 0
```

Validation: all 25 verifier scenarios passed, including sanitizer checks with leak detection disabled. Phase 1 checks still pass. Your Mac run and CMake execution remain unverified.

Read `docs/phase2-walkthrough.md` alongside the code. It explains each check and collects the review questions at the end. `docs/phase2-development-log.md` contains the implementation and test record to merge into your main log. Nothing has been pushed to GitHub.

You

output = expected output. success

Worked for 23s

ChatGPT

Logged **P2-M01: PASS**—your local demo accepted the stable assignment and rejected the unstable assignment with the expected blocking pair.

Update your documentation with phase2-development-log.md .

Next, run the separate verifier tests:

```
c++ -std=c++17 -Wall -Wextra -Wpedantic -I algorithm/include
algorithm/src/stability.cpp tests/stability_tests.cpp -o build/stability_tests
./build/stability_tests
```

Expected:

```
Passed 25 verifier scenarios and blocking-pair ID check
```

phase2-development-log.md
Document

Take your curiosity further
Scan to get the ChatGPT app

You

Passed 25 verifier scenarios and blocking-pair ID check - success

Worked for 24s

ChatGPT

Logged **P2-M02: PASS**—all 25 verifier scenarios and the blocking-pair ID check passed on your machine.

Replace your local copy with the updated phase2-development-log.md .

Phase 2's implementation and initial execution checks are complete. Phase 3 will cover randomized cases, exhaustive checks on small inputs, and performance measurements.

phase2-development-log.md
Document

You

done

ChatGPT

Phase 2's documentation is up to date. Both the demo and the 25-scenario verifier suite have passed locally.

Next is **Phase 3: broader testing**—randomized inputs, exhaustive checks on small cases, and performance measurements.

You

Begin

Worked for 3m 36s

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Phase 3 is implemented, and all checks passed here.

Download mentor-matching-system-phase3-update.zip

Copy its contents into your existing project root, preserving paths. Replace `CMakeLists.txt`. Your existing logs, screenshots, and production code remain unchanged.

The new suite covers:

Check	Coverage
Exhaustive testing	Every preference profile with up to three mentors: **46,973 profiles and 281,723 candidate assignments**.
Independent comparison	Verifier results checked against a separate linear-search implementation.
Mentor-optimality	Computed assignments compared with every stable alternative in the exhaustive cases.
Randomized testing	**1,000 repeatable cases**, seed 4330.
Invalid-data mutations	**866 cases × four rejection checks**.
Heavy competition	Eight cases with known exact proposal counts.

Run the correctness suite in your VS Code terminal:

```
mkdir -p build
c++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I algorithm/include
algorithm/src/matching.cpp algorithm/src/stability.cpp
tests/core_validation.cpp -o build/core_validation
./build/core_validation
```

Expected output:

Take your curiosity further
Scan to get the ChatGPT app

```
Exhaustive: 46973 preference profiles; 281723 candidate assignments passed
Randomized: 1000 cases passed (seed 4330)
Mutations: 866 cases x 4 rejection checks passed
Contention: 8 shared-ranking cases passed
ALL PHASE 3 CORRECTNESS CHECKS PASSED
```

Then run the benchmark:

```
./build/core_validation --benchmark
```

It reports proposal counts and median matching/verifying times. **Your timings will differ from mine.**

The package includes measured results, a development-log entry, and `docs/phase3-walkthrough.md` explaining the tests with review questions at the end.

The full suite also passed sanitizer checks here, with leak detection disabled. Your local execution remains pending; CMake/CTest wasn't executed here. No GitHub changes were pushed.

You



expected out = putput and i ran the benchmark- here is the result

Worked for 42s >

ChatGPT

Phase 3's local correctness checks passed, and the benchmark completed successfully. I logged both results.

Your largest cases show:

Rankings	Mentors / mentees	Proposals	Matching	Verification
Random	1,000 / 1,000	7,398	1.764 ms	2.863 ms

Take your curiosity further
Scan to get the ChatGPT app

Rankings	Mentors / mentees	Proposals	Matching	Verification
Shared	1,000 / 1,000	500,500	5.707 ms	5.811 ms
Shared, excess mentors	2,000 / 1,000	1,500,500	13.857 ms	12.373 ms

Shared rankings create more competition and proposals. The counts match the expected formula and stay below mn. These timings describe the tested cases, rather than guaranteed performance.

Replace your local phase3-development-log.md , and keep this screenshot in `docs/screenshots/` as benchmark evidence.

The core matching milestone now passes its planned execution checks. Code review remains before we move into database design.

phase3-development-log.md
Document

You

screenshot suploaded and phase 3 development log updated

ChatGPT

Phases **1–3 are implemented, tested locally, and documented**. The core matching milestone is established.

Before Phase 4, we'll review the matching and verification code together so you understand how it works. I'll explain it in connected sections and keep review questions bundled at the end.

You

Lets do it

Worked for 8s >

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

The core has two separate jobs: `matchMentors()` constructs assignments; `verifyStability()` judges an assignment. Phase 3 tests both.

Open the files in VS Code as you follow this walkthrough.

1. The header describes the data contract

In `algorithm/include/matching.hpp`:

```
using PreferenceLists = std::vector<std::vector<std::size_t>>;
```

This is a type alias for a vector of preference lists. Each row belongs to one participant; its entries are opposite-side IDs, ordered from best to worst.

```
PreferenceLists mentors = {
    {2, 0, 1},
    {1, 2, 0}
};
```

Here, Mentor 0 prefers Mentee 2, then Mentee 0, then Mentee 1.

`std::size_t` is an unsigned integer type used for sizes and indices. These IDs are positions within one matching run, rather than permanent user IDs.

The output is:

```
struct MatchingResult {
    std::vector<std::optional<std::size_t>> mentor_to_mentee;
    std::vector<std::optional<std::size_t>> mentee_to_mentor;
    std::size_t proposal_count = 0;
};
```

An `optional` contains either an ID or no value. That distinguishes **Mentee 0** from **no mentee**.

We store each assignment in both directions:

```
mentor_to_mentee[2] = 0;
mentee_to_mentor[0] = 2;
```

Take your curiosity further
Scan to get the ChatGPT app

Both lines describe the same pair. This makes partner lookup immediate, but requires us to keep both records consistent.

The function declaration is:

```
MatchingResult matchMentors(
    const PreferenceLists& mentor_preferences,
    const PreferenceLists& mentee_preferences
);
```

`&` passes the existing lists by reference, avoiding copies. `const` prevents the function from modifying them through those references.

The `.hpp` declares this interface; `matching.cpp` supplies its implementation.

2. Validate preferences before indexing

At the start of `matching.cpp`, `validatePreferences()` checks each row:

- Its length equals the opposite group's size.
- Every ID is within range.
- No ID appears twice.

Together, these conditions guarantee that each person ranks everyone exactly once.

```
std::vector<bool> seen(opposite_count, false);
```

`seen[id]` records whether an ID has appeared in the current row. Importantly, we check the ID's range **before** accessing `seen[id]`.

Malformed inputs throw `std::invalid_argument`, stopping that matching request with an explanation.

The helper sits inside:

```
namespace mentoring {
namespace {
    // Helper function
}
}
```

`mentoring` groups our project's names. The unnamed namespace keeps this helper local to this implementation file.

Take your curiosity further
Scan to get the ChatGPT app

3. Prepare the matching state

Inside `matchMentors()`:

```
const std::size_t m = mentor_preferences.size();
const std::size_t n = mentee_preferences.size();
```

The outer vector sizes determine the participant counts. We reject $m < n$, then validate both sets of preferences.

Next:

```
MatchingResult result;
result.mentor_to_mentee.resize(m);
result.mentee_to_mentor.resize(n);
```

The optional entries initially contain no values, so everyone starts unassigned.

If there are no mentees, we return immediately.

The next structure is especially important:

```
PreferenceLists mentee_rank(n, std::vector<std::size_t>(m));
```

A preference list answers:

> Who occupies this preference position?

A rank lookup answers:

> What position does this person occupy?

For a mentee whose list is `{2, 0, 1}`:

Mentor ID	Rank
2	0
0	1

Take your curiosity further
Scan to get the ChatGPT app

Mentor ID	Rank
1	2

We construct that reverse lookup with:

```
mentee_rank[mentee][mentee_preferences[mentee][position]] = position;
```

Now comparing two mentors takes $O(1)$ time. Smaller rank means more preferred.

The remaining state is:

```
std::vector<std::size_t> next_proposal(m, 0);
std::queue<std::size_t> available;
```

`next_proposal[a]` remembers where Mentor a should look next in their list.

`available` contains unassigned mentors who still have someone to approach. Initially, all mentors enter the queue.

4. The proposal loop implements deferred acceptance

Each iteration removes one mentor from the queue:

```
const std::size_t mentor = available.front();
available.pop();
```

Then it selects their next choice:

```
const std::size_t mentee =
    mentor_preferences[mentor][next_proposal[mentor]];

++next_proposal[mentor];
++result.proposal_count;
```

We advance the position immediately, regardless of acceptance. This prevents repeated proposals.

The mentee's current partner is:

```
const auto current = result.mentee_to_mentor[mentee];
```

`auto` lets the compiler infer the optional's type. This variable is a copy of the current assignment.

Take your curiosity further
Scan to get the ChatGPT app

There are three possible outcomes:

Situation	Action
Mentee is unassigned	Tentatively assign the proposer.
Mentee prefers the proposer	Release the previous mentor and assign the proposer.
Mentee prefers their current mentor	Reject the proposer.

The preference comparison is:

```
mentee_rank[mentee][mentor] < mentee_rank[mentee][*current]
```

`*current` retrieves the mentor ID inside the optional. We reach this comparison only after establishing that `current` contains a value.

When replacing a mentor:

```
result.mentor_to_mentee[*current].reset();
```

`reset()` clears the previous mentor's assignment. Both directions are then updated for the new pair.

A rejected or displaced mentor returns to the queue only if:

```
next_proposal[mentor] < n
```

For a displaced mentor, we perform the equivalent check using `*current`.

This condition keeps exhausted mentors out of the queue. It also ensures that the next preference access is within bounds.

5. Why the loop finishes with a stable result

Each mentor proposes to each mentee at most once, giving at most:

$$mn$$

proposals. Eventually, the queue empties.

Take your curiosity further
Scan to get the ChatGPT app

Suppose a mentor prefers some mentee over their final partner—or remains unassigned. That mentor must already have approached that mentee.

The mentee rejected them or later replaced them with someone preferred. Since a mentee's tentative partner only improves, their final partner is preferred to that mentor.

Therefore, they cannot form a blocking pair.

Under our complete-ranking assumptions, every mentee finishes assigned, with exactly $m - n$ mentors unassigned. Mentor-proposing deferred acceptance also gives each mentor their best attainable outcome **among stable matchings**.

6. The verifier checks the result independently

Open `algorithm/src/stability.cpp`.

The verifier does not call `matchMentors()`. It accepts supplied preferences and assignments—even assignments constructed manually.

It checks things in this order:

1. Preferences satisfy our input rules.
2. Assignment vectors have the correct sizes.
3. Partner IDs are valid and every mentee is assigned.
4. Both assignment directions agree.
5. No blocking pair exists.

The order matters: **validate IDs before using them as indices.**

Reciprocal checks also prevent duplicate assignments. If two mentors both claim Mentee 0, her single partner entry cannot agree with both.

After structural validation, the verifier builds rank tables for both sides and examines every nonassigned pair:

```
const bool mentor_prefers = !mentor_partner[mentor]
    || mentor_rank[mentor][mentee]
       < mentor_rank[mentor][*mentor_partner[mentor]];
```

The mentor prefers the proposed alternative if they are unassigned or rank that mentee above their current partner.



Take your curiosity further

Scan to get the ChatGPT app



`||` short-circuits: when the mentor is unassigned, the right-hand expression is skipped. We never dereference their empty optional.

The mentee's condition is:

```
const bool mentee_prefers = mentee_rank[mentee][mentor]
    < mentee_rank[mentee][*mentee_partner[mentee]];
```

Every mentee has already been confirmed assigned.

Only this combination creates instability:

```
if (mentor_prefers && mentee_prefers)
```

The verifier returns the first blocking pair it finds. Otherwise, it returns `Stable`.

`isReasonable()` is the Boolean convenience function: only `Stable` produces `true`.

7. Phase 3 challenges both implementations

Our tests go beyond feeding matcher results into the verifier.

For small inputs, they enumerate every complete assignment and compare the verifier against a separate reference that searches preference lists directly. They also compare the matcher's result against every stable alternative to check mentor-optimality.

Random tests cover larger inputs. Mutations test rejection of selected invalid data. Benchmarks measure execution time and proposal counts.

For equal-sized groups, both production functions use $O(n^2)$ time and space. Even when matching needs relatively few proposals, reading complete rankings and constructing rank tables still requires quadratic work.

Your passing results provide strong evidence within the tested scope; the correctness argument explains why the algorithm works generally under our assumptions.

Review questions—collected here for whenever you're ready:

1. Why does an assignment need `optional` when ID 0 is valid?
2. What is the difference between a preference list and an inverse-rank table?
3. Why advance `next_proposal` even when a proposal succeeds?
4. What must be updated when a mentee replaces their current mentor?
5. Why can an exhausted, unassigned mentor safely leave the queue?



Take your curiosity further

Scan to get the ChatGPT app



6. Why must both people prefer each other for a pair to block?
7. How do reciprocal assignment checks detect duplicate partners?
8. Why should the verifier accept a different stable matching?
9. How does Phase 3 avoid relying exclusively on the production verifier?
10. Why can few proposals still accompany quadratic total work?

You

Then we're ready for phase 4

Worked for 34s

ChatGPT

Yes—Phase 4 is database design. We need to settle SQLite versus Supabase before writing the schema.

My recommendation remains **SQLite for a focused C++ class project**. It runs inside the backend and stores the database in a file, without a separate database server. Supabase provides hosted PostgreSQL and authentication, which would help with individual user accounts. www2.sqlite.org +2

The database needs to represent:

Data	Purpose
Users	Identity, administrator privileges, and active/inactive status.
Matching rounds and participants	Who participates as a mentor or mentee in each round.
Preference versions	Complete rankings, including previous versions.
Change explanations	Required notes accompanying preference changes.
Matching runs	Which exact participants and preference versions produced a result.
Assignments	Mentor–mentee pairs belonging to each matching run.

Take your curiosity further
Scan to get the ChatGPT app

Data	Purpose
Verification results	Whether a run passed validation and stability checks.
Ratings	Feedback tied to an assignment, accessible only to administrators.

Preserving the inputs to each matching run is essential. Otherwise, someone changing their preferences would make it impossible to reconstruct why an earlier assignment was considered stable.

Separating a person's identity from their role in a round also lets a mentee become a mentor later without losing their history.

The database will enforce structural rules such as unique ranks and one-to-one assignments. The C++ verifier will determine stability.

Which database do you want to commit to: SQLite or Supabase? That choice affects the schema implementation, authentication, and deployment setup.

You

SQLite

Worked for 5m 59s

ChatGPT

Phase 4's SQLite schema is implemented and ready for review. All 31 database tests passed here.

Download mentor-matching-system-phase4-update.zip

Copy its contents into your project root, preserving paths. Your C++ core, CMake configuration, existing logs, and screenshots remain unchanged.

The schema stores users, round-specific roles, versioned preferences, change explanations, matching runs, assignments, verification results, and ratings.

Three design defaults are documented for your review:

Take your curiosity further
Scan to get the ChatGPT app

- **Roster freezes before preference submission.** Later additions, removals, and promotions use a new round.
- **Submitted rankings remain historical records.** Changes create a new version with an explanation.
- **Ratings use a provisional 1–5 scale**, with one submission per partner per assignment.

Changing preferences marks earlier inputs as outdated—it does not automatically declare the assignments unstable.

Run these from your VS Code terminal:

```
python3 database/demo.py
python3 database/tests/test_schema.py
```

The demo should end with:

```
DATABASE DEMO PASSED
```

The tests should end with:

```
Ran 31 tests ...
OK
```

These Python scripts are **database test utilities**. The backend remains C++; connecting it to SQLite is still pending. The demo stores a known example rather than calling the C++ matcher.

Read `database/README.md` for the schema explanation and bundled review questions. `docs/phase4-development-log.md` records the checks. Authentication and admin-only rating access remain API responsibilities.

You



Ran 31 tests in 0.043s

Worked for 28s

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

All 31 schema tests passed locally, and the screenshot confirms the database demo passed too.

Logged **P4-M01 and P4-M02: PASS**, including your reported **0.043-second** test time.

Update your local phase4-development-log.md , and keep `database test.png` in `docs/screenshots/` .

Phase 4's schema checks are complete. The C++ connection, authentication, and API remain pending.

phase4-development-log.md
Document

You

photo uploaded and development log updated.

ChatGPT

Phase 4's local tests and documentation are up to date.

Next is **Phase 5: the C++ API**—connecting SQLite to the matching core and exposing operations for preferences, administrator-triggered matching, assignments, and ratings, with authentication and role restrictions.

You

Begin

Worked for 21m 37s

ChatGPT

Phase 5 is implemented. All 33 HTTP integration tests and all five CMake test suites passed here.

Download mentor-matching-system-phase5-update.zip

Take your curiosity further
Scan to get the ChatGPT app

Copy its contents into your project root, preserving paths. Replace `CMakeLists.txt`; your existing logs and screenshots remain unchanged.

The API now supports:

- Administrator-created accounts, login, logout, and password changes.
- Round and participant management.
- Participants submitting their own preferences.
- Administrator-triggered matching using the **real C++ matcher and verifier**, saved to SQLite.
- Rechecking existing assignments after preference changes.
- Participant-specific assignment views and administrator-only rating reads.

On your Mac, the new build requires **OpenSSL 3**. If needed, install it through Homebrew:

formulae.brew.sh

```
brew install openssl@3
```

Then, from your project root:

```
bash backend/build.sh
python3 backend/tests/test_api.py
```

Expected ending:

```
Ran 33 tests ...
OK
```

The tests start and stop their own server and use temporary databases. **You don't need to start a server manually.**

Read `backend/README.md` for setup and the code walkthrough, and `backend/API.md` for the endpoint contract. Review questions are bundled at the end of the README.

Your Mac build remains unverified. The server is configured for localhost development; nothing has been deployed or pushed to GitHub.

You

ran 33 tersts in 4.736s
files have been uploaded

Take your curiosity further
Scan to get the ChatGPT app

Worked for 36s >

ChatGPT

Recorded **33 tests in 4.736 seconds** in phase5-development-log.md .

Did the final summary end with **`0K`** ? Once confirmed, I'll mark the local run as passed.

phase5-development-log.md
Document

You

yes it did

Worked for 24s >

ChatGPT

Logged **P5-M01: PASS**—all 33 API integration tests passed locally in **4.736 seconds**, ending with `0K` .

Replace your local phase5-development-log.md .

Phase 5's local execution checks are complete. Next is **Phase 6: the minimal UI**, guided by your design ideas.

phase5-development-log.md
Document

You

oaky, its been replaced

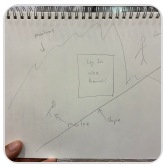
ChatGPT

Phase 5's local test result is now documented.

Take your curiosity further
Scan to get the ChatGPT app

For **Phase 6**, send your UI design ideas or sketches. We'll use those to shape the minimal interface for login, mentor/mentee preferences and assignments, and administrator controls.

You



Here is the idea. Mountainous background, then a slope close up, mentee at the bottom looking up, mentor at the top. When you click one or the other, it pops up the login center. If you click the top of the mountain in the background, the server admin can login. If it's a mentor, the mentee disappears and is replaced with the list of mentees to build the preference list. Vice versa if a mentee logs in. For server admin, both are replaced with a list of lists, mentors and their preferences and mentees and their preferences. I'm just thinking about design

ChatGPT

The mountain metaphor fits the system: the mentee is beginning the climb, while the mentor is farther along and can guide them. Keeping that landscape across the different screens would give the project a recognizable identity.

I understand your interactions like this:

State	What appears
Before login	Mountain background, close-up slope, mentee below and mentor above.
Click either person	A centered login panel appears over the landscape.
Mentor logs in	The mentee figure gives way to a ranked list of mentees; the mentor remains.

Take your curiosity further
Scan to get the ChatGPT app

State	What appears
Mentee logs in	The mentor figure gives way to a ranked list of mentors; the mentee remains.
Administrator logs in through the background summit	Both figures give way to mentor and mentee preference panels, alongside matching controls.

A few refinements would help preserve the idea while making it usable:

- **Label the clickable figures.** “Mentor” and “Mentee,” with a subtle hover/focus effect, make the interaction discoverable.
- **Keep the summit entrance, but provide a small “Administrator” link too.** The mountain can be a playful shortcut.
- **Place ranking lists on readable panels.** Keep the diagonal slope behind them; names and controls should remain horizontal.
- **Support dragging and up/down buttons.** Both should change the same numbered preference order, including on mobile.
- **Use a short transition after login.** The opposite figure fades into the preference panel, preserving the spatial relationship you sketched.

One important distinction: **clicking a figure selects the intended entrance; the authenticated account determines access.** Clicking the summit cannot grant administrator privileges. Also, mentor/mentee roles belong to a round, so the interface should reflect the selected round.

Your administrator view introduces one addition to the current API: **an admin-only endpoint for reading participants’ submitted preference lists.** Right now, the preference-reading endpoint returns only the caller’s own list.

For the visual direction, I’d keep the mountain illustration restrained—muted colors, simple figures, generous space—and let the ranking panels carry the detail. That would preserve your minimalistic goal.

We can keep this at the design stage while you develop the idea further.

You

That sounds good. build it

Take your curiosity further
Scan to get the ChatGPT app

Worked for 6m 23s >

ChatGPT

Your mountain interface is built, with “Ascent” as a working name.

Download mentor-matching-system-phase6-update.zip

Copy its contents into your project root, preserving paths. Replace the included backend files too.

To try the design:

```
python3 -m http.server 5173 --bind 127.0.0.1 --directory frontend
```

Open http://127.0.0.1:5173. Click the mentor, mentee, or summit, then choose **“Explore this role with sample data.”**

It includes:

- Mountain landscape and clickable figures.
- Centered login panel.
- Role-specific rankings with dragging and arrow controls.
- Administrator preference overview and assignment button.
- Live API connections for preferences, matching, and feedback.

Browser checks passed for the mentor and administrator demo flows. **All 34 API tests passed.** Full live browser integration and mobile verification remain pending.

See `frontend/README.md` for live-backend setup and the remaining UI work. Nothing has been deployed or pushed to GitHub.

You

I like it, but the mentor and mentee sit on the path. I get the need to be reactive to window size, but this will be presented on my comp so need for mobile responsive. the texts can be a bit bigger too.

increase the size of:

"A little guidance. A long way"

Take your curiosity further
Scan to get the ChatGPT app

"Growth is a shared journey"

Center "choose your place on the mountain to begin" (remove the period as well) so it's in between the mentor and mentee but still below "Rise Together"

remove:

"A little guidance a long way"

"different paths. shared purpose"

the mentee can be on the line as well, and increase the text underneath both the mentor and mentee

Worked for 3m 40s

ChatGPT

Updated both figures to stand on the path, enlarged the text, centered the instruction without its period, and removed the two taglines. Desktop layout verified.

Download the updated files

Replace `frontend/index.html`, `frontend/styles.css`, and `frontend/app.js`. No backend rebuild needed.

You

move the"choose your place on the mounta to begin to the left and up and change the font color to be slightly lighter than the color of find oyur footing rise together. the c should be under the h, and its aligned with the middle of the mentor. Include the lifting function when hovering over either as well.

You stopped after 33s

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

I'll move the instruction up and left, align its "C" beneath the "h" in "Rise together," and use a slightly lighter shade than the headline. I'll also restore the hover lift on both figures.

Ran 3 commands

You

move the"choose your place on the mounta to begin to the left and up and change the font color to be slightly lighter than the color of find oyur footing rise together. the c should be under the h, and its aligned with the middle of the mentor. Include the lifting function when hovering over either as well.

also, remove the administrator button at the top of mountain, the one on the buttom right is great

Worked for 1m 19s

ChatGPT

Updated the instruction's alignment and lighter color, restored hover lift for both figures, and removed the summit administrator button. The bottom-right link remains.

Download updated files

Replace `frontend/index.html`, `frontend/styles.css`, and `frontend/app.js`. No backend rebuild needed. JavaScript syntax passed; visual verification remains pending.

You

okay lets continue

Worked for 28s

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Next is **Phase 7: live integration**—testing the mountain UI with real accounts, SQLite preferences, and C++ matching.

First, run from your project root:

```
bash backend/build.sh
python3 backend/tests/test_api.py
```

Expected ending:

```
Ran 34 tests ...
OK
```

These tests create temporary databases. After they pass, we'll initialize your persistent application database and administrator account, then test the full mentor–mentee workflow through the interface.

Send the final test summary when it finishes.

You

lets go back to the UI for a moment. we need exit buttons for all prompt boxes so a person can get out and back to pervious page. also, align the "choose your" under rise together -- and make it a darker green than find your footing

Worked for 1m 12s

ChatGPT

Updated:

- Added close/back buttons to login, matching confirmation, and every role workspace.
- Aligned "Choose your..." directly beneath the start of "Rise together."
- Changed the instruction to a darker green than the headline.

Download updated files

Take your curiosity further
Scan to get the ChatGPT app

Replace the three frontend files: `index.html`, `styles.css`, and `app.js`. No backend rebuild needed.

Syntax checks passed; visual verification remains pending.

You

Great. now lets move onto phan 7: integration

Worked for 6m 14s

ChatGPT

Phase 7's integration tools are ready. All 34 API tests and the new SQLite persistence workflow passed here. Browser verification remains pending.

Download mentor-matching-system-phase7-update.zip

Copy its contents into your project root, then run:

```
bash backend/build.sh
python3 backend/tests/test_api.py
python3 tests/integration/test_live_workflow.py
```

Both test suites should end with `OK`.

Then stop any existing frontend/backend servers and run:

```
python3 tools/phase7_demo.py
```

Open http://127.0.0.1:5173. Login credentials are generated in `build/phase7-demo/accounts.json`.

This starts a separate live cohort with three mentors and two mentees. Use **Sign in**, then follow `docs/phase7-integration.md` to test preferences, matching, feedback, and persistence.

You



Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

The launcher couldn't start because port 8080 or 5173 is already in use, likely by an earlier server.

1. Switch to the other terminal labeled **Python** and press **Ctrl+C** if it's running a server.
2. Check both ports:

   ```
   lsof -nP -iTCP:8080 -sTCP:LISTEN
   lsof -nP -iTCP:5173 -sTCP:LISTEN
   ```

   If either lists a process, stop that server in its terminal. Then retry:

   ```
   python3 tools/phase7_demo.py
   ```

Your screenshot also shows:

- **API tests:** 33 passed in **4.777s**.
- **Integration workflow:** passed in **2.893s**.
- **ResourceWarning:** cleanup warning; both suites still ended with `OK`.

One discrepancy needs fixing before browser testing: Phase 6 should have increased the API suite to **34 tests**. Copy these two files from the latest Phase 6 ZIP into your project:

```
backend/src/service.cpp
backend/tests/test_api.py
```

Then rebuild and rerun:

```
bash backend/build.sh
python3 backend/tests/test_api.py
```

Expected: **34 tests**, ending with `OK`.

You

give me the files from phase 6

ChatGPT

Download the latest Phase 6 files

Take your curiosity further
Scan to get the ChatGPT app

For the API update, replace:

- `backend/src/service.cpp`
- `backend/tests/test_api.py`

Then run:

```
bash backend/build.sh
python3 backend/tests/test_api.py
```

Expected: **34 tests**, ending with `OK`.

You



ChatGPT

The launcher started successfully. Both servers are running at http://127.0.0.1:5173.

The commands typed below “Live integration ready” haven’t executed—the launcher is occupying that terminal.

To verify the updated backend:

1. Press **Ctrl+C**.
2. Run:

   ```
   bash backend/build.sh
   python3 backend/tests/test_api.py
   ```

3. After **34 tests → OK**, restart your existing cohort:

   ```
   python3 tools/phase7_demo.py --resume
   ```

Then open the site, find the generated password in `build/phase7-demo/accounts.json`, and sign in through the bottom-right **Administrator** link using `admin@ascent.test`. Leave the launcher terminal running.

Take your curiosity further
Scan to get the ChatGPT app

You

do i run both in python?

ChatGPT

Run both commands in the **VS Code terminal**, where you see the `%` prompt—not inside Python.

First press **Ctrl+C** to stop the launcher, then enter:

```
bash backend/build.sh
```

When it finishes, enter:

```
python3 backend/tests/test_api.py
```

The first builds the C++ backend using Bash. The second runs the tests using Python.

You



i still get address in use but the 34 tests ran successfully

Worked for 1m 36s

You



this is what I saw when I hit ctrl c- sys building for macos -26.0, but linkin giwth dylib?

Worked for 1m 25s

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

That's a compatibility warning, not the "address in use" error.

- Your build targets **macOS 26.0**.
- OpenSSL's `libcrypto.3.dylib` —a shared cryptography library—was built for **27.0**.
- The traceback below it came from **Ctrl+C interrupting the tests**.

Your later **34 tests → OK** remains valid. To check the warning, run:

```
sw_vers -productVersion
```

Send me that output before we change any macOS settings.

I also fixed a reproducible **port-restart bug**. Download the corrected Phase 7 update and copy its contents into your project, replacing the included files.

Then run each command separately:

```
bash backend/build.sh
python3 tests/integration/test_live_workflow.py
python3 tools/phase7_demo.py --resume
```

The integration suite should now report **3 tests → OK**. All three and the 34 API tests passed here. Your existing cohort database is preserved.

You

27.0 for produtverion

ChatGPT

Your Mac runs **macOS 27.0**, matching the OpenSSL library. The warning comes from the compiler targeting the older **26.0** version.

For this build, set the target to your actual OS:

```
MACOSX_DEPLOYMENT_TARGET=27.0 bash backend/build.sh
```

Then run:

Take your curiosity further
Scan to get the ChatGPT app

```
python3 tests/integration/test_live_workflow.py
```

After **3 tests → OK**, resume:

```
python3 tools/phase7_demo.py --resume
```

Use the corrected Phase 7 files from my previous message first.

You

wrongaccount@Nots-MacBook-Pro mentor-matching-system % bash backend/build.sh
ld: warning: building for macOS-26.0, but linking with dylib '/opt/homebrew/opt/openssl@3/lib/libcrypto.3.dylib' which was built for newer version 27.0

ChatGPT

You ran the original command. Use this full line to target macOS 27.0:

```
MACOSX_DEPLOYMENT_TARGET=27.0 bash backend/build.sh
```

Once it finishes and the `%` prompt returns, run:

```
python3 tests/integration/test_live_workflow.py
```

You



to stop the old server, do i have to go to python and hit ctrl c?

ChatGPT

Yes—select the VS Code terminal running the old server (possibly labeled "Python") and press Ctrl+C. You should see the `%` prompt return.

Take your curiosity further
Scan to get the ChatGPT app

Then run:

```
python3 tools/phase7_demo.py --resume
```

If port **8080** is still unavailable, run this and send the output:

```
lsof -nP -iTCP:8080 -sTCP:LISTEN
```

Your screenshot confirms **all 3 integration tests passed in 3.174 seconds**, and the latest build shows no macOS version warning.

You



ChatGPT

It's running successfully now. Leave that terminal open.

1. Open http://127.0.0.1:5173.
2. In VS Code, open `build/phase7-demo/accounts.json` to find the password.
3. Click **Administrator** at the bottom right.
4. Sign in with `admin@ascent.test` and that password—not sample-data mode.

You should see **three mentors, two mentees, and Assign disabled** because their preferences haven't been submitted yet.

You

where can i find the password?

ChatGPT

In **VS Code**, press **Cmd + P**, type:

```
build/phase7-demo/accounts.json
```

Take your curiosity further
Scan to get the ChatGPT app

Press **Enter**. Copy the value beside `"password"` **without the quotation marks**.

Use it with admin@ascent.test.

You

I'm i- 3 mentors and 2 mentees

ChatGPT

You're in—the live administrator view shows the expected **3 mentors and 2 mentees**.

Next, sign out and submit preferences for each account using the same password:

Entrance	Email
Mentor	`alex@ascent.test`
Mentor	`morgan@ascent.test`
Mentor	`sam@ascent.test`
Mentee	`jordan@ascent.test`
Mentee	`casey@ascent.test`

For each account, leave the default ranking order, click **Submit preferences**, then sign out.

Finally, sign back in as admin@ascent.test. **Assign mentoring partners** should become enabled once all five lists are submitted. Keep the server terminal running throughout.

You

It worked!

ChatGPT

The live preference-submission flow worked.

Take your curiosity further
Scan to get the ChatGPT app

Now, as administrator, click **Assign mentoring partners** and confirm. With the default rankings, expect:

Mentor	Mentee
Alex Rivera	Jordan Ellis
Morgan Chen	Casey Brooks
Sam Taylor	Unassigned

Then sign in as Jordan and Casey to check their assigned partners and submit feedback. Sign in as Sam to confirm he sees **“currently unassigned.”**

You

I signed in as Sam and he is currently unassigned. I also submitted feedback

ChatGPT

Confirmed: **Sam correctly appears unassigned**, and you successfully submitted feedback through the live UI.

The final persistence check:

1. Stop the launcher with **Ctrl+C**.
2. Restart it:

   ```
   python3 tools/phase7_demo.py --resume
   ```

3. Refresh the browser and sign in again.
4. Confirm the saved preference order and assigned partner remain.

That will verify the live workflow survives a server restart.

Take your curiosity further
Scan to get the ChatGPT app

You

where would the feedback be provided?

ChatGPT

Feedback is **submitted in the assigned mentor's or mentee's workspace**, beneath their partner's name, using the rating and comment fields.

It's saved in SQLite and is **visible only to administrators through the API**. We haven't added a feedback-viewing panel to the administrator UI yet, so it won't appear on that screen.

That panel is the next UI addition we need.

You

I restarted it and launched but I only see preference ordered saved, not the assigned partner. I dont think that saved

Yes! add that panel. it should be separate from the assignment panel, beneath it with the same dimensions. It should be Feedback From: then split between Mentors and Mentees just like the assignment panel. then see their feedback

Worked for 5m 9s

ChatGPT

Added the feedback panel and fixed the administrator view's failure to reload saved assignments.

"Feedback From:" now appears beneath the main panel, with matching width/minimum height and separate **Mentors** and **Mentees** columns. Each entry shows the author, partner, rating, and comment.

Download updated Phase 7 files

Replace these three files:

- `frontend/index.html`
- `frontend/styles.css`

Take your curiosity further
Scan to get the ChatGPT app

- `frontend/app.js`

Then press **Cmd+Shift+R** in the browser and sign in again. **No rebuild, database reset, or new matching run is needed.**

Six frontend checks and all three persistence/restart tests passed. Your local visual check remains pending.

You

great it works! lets document all these testings and errors and corrections then move on to the next phase

Worked for 12s

You

Continue this chat

Worked for 6m 9s

ChatGPT

The tests, errors, and corrections are documented, and Phase 8—full-system testing—is ready.

Download Phase 8 update

It includes the consolidated test/issue register, Phase 7 closeout, updated roadmap, and a test runner. Your earlier logs and screenshots stay intact.

Copy the folders into your project root. Stop the running launcher with **Ctrl+C**, then run:

```
python3 tools/run_full_system_tests.py
```

All executed checks passed here. Your Mac run and remaining manual browser checks are pending. Send the final summary when it finishes; deployment remains Phase 9.

Take your curiosity further
Scan to get the ChatGPT app

You



Worked for 56s

ChatGPT

Every executed check passed. “PARTIAL” means the six frontend controller tests were skipped because Node isn’t installed.

In your VS Code terminal, install Node through Homebrew: formulae.brew.sh

```
brew install node
```

Then run only the skipped suite:

```
node --test tests/integration/frontend_restore.cjs
```

Expected: **6 passed, 0 failed**. No need to repeat the passing suites.

I recorded your results in the updated Phase 8 documentation . Browser acceptance checks remain pending.

You

all 6 tests passed

Worked for 32s

ChatGPT

Logged **6/6 frontend checks: PASS**. Every automated suite has now passed locally.

Updated Phase 8 documentation

Next, finish the browser checks. Start:

```
python3 tools/phase7_demo.py --resume
```

Take your curiosity further
Scan to get the ChatGPT app

After refreshing the browser, verify:

1. Admin sees saved assignments and feedback without running Assign again.
2. An assigned participant sees their saved partner.
3. Sam remains unassigned.
4. Participants cannot see the administrator feedback panel.

Report any mismatch; otherwise, we'll continue with cancellation and error-handling checks.

You

from updated 8 documentation, which file is new?

ChatGPT

For the latest **6/6 tests passed** entry, no new file was added. These two documents were updated:

- `docs/phase8-development-log.md`
- `docs/testing-and-issue-register.md`

Replace those two files from the latest ZIP. No code changes or rebuild needed.

You

1 selection

verified

Worked for 24s

ChatGPT

Logged **all four browser checks: PASS** in the updated Phase 8 documentation . Replace the same two documentation files.

Next, verify:

1. **Login cancellation:** each entrance closes using ×, Back, and Escape.
2. **Matching cancellation:** open Assign, then cancel. Existing assignments remain unchanged.

Take your curiosity further
Scan to get the ChatGPT app

3. **Wrong password:** login is rejected; the correct password still works afterward.
4. **Preference validation:** change a saved ranking and submit without an explanation—it should reject the update. Then restore the original order without submitting.

Report which pass or any unexpected behavior.

You

all passed- lets deploy now

Worked for 10m 47s

ChatGPT

Render is connected, and the deployment package is ready. The app isn't live yet.

Download Phase 9 deployment files

Copy these into your project, then commit and push to `ninonyc/mentor-matching-system`. Render needs those files in GitHub before it can build; I don't have repository write access here.

Seven frontend checks and six deployment-helper tests passed. The Docker build remains pending.

SQLite persistence requires a **paid Render service and disk**; no paid resources have been created.

Render Docs

Tell me when the files are pushed, and we'll continue with Render's service setup.

You

files have been pushed to github

Worked for 2m 1s

ChatGPT

Render shows one workspace: **mentor-matching**. Should I use it for this deployment?

Take your curiosity further
Scan to get the ChatGPT app

You

yes

Worked for 21s >

ChatGPT

Render couldn't access the repository, `main` branch, or `render.yaml`. Nothing has been deployed or charged.

Please check:

1. GitHub's render.yaml opens and contains the deployment configuration.
2. In Render account settings, connect GitHub and grant access to **ninonyc/mentor-matching-system**.

Tell me when that's done, and I'll retry.

You



ChatGPT

Yes—your screenshot confirms **render.yaml is on GitHub**.

Next, connect GitHub in Render settings and grant Render access to **ninonyc/mentor-matching-system**. Keep the repository private if you prefer.

Once connected, tell me and I'll retry the deployment plan.

You

done

Worked for 15s >

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

GitHub access works now. Render's next blocker is **payment information**, required for the service and persistent database storage.

Add a payment method in your workspace's **Billing** section in the Render Dashboard, then tell me when done.

No service has been created. I'll show you the deployment plan and costs before applying it.

You

isnt it free?

Worked for 9s

ChatGPT

Render has a free tier, but our current setup requires paid hosting. I should have made that clear earlier.

Your app stores accounts, assignments, and feedback in SQLite. Render's free tier cannot attach persistent storage, so that data would disappear when the service restarts, redeploys, or goes idle.

Render Docs

We can either:

- **Keep SQLite and use paid hosting** to preserve saved data.
- **Keep hosting free and adapt the app to an external database**, which requires additional backend work.

Nothing has been deployed or charged. Would you prefer we pursue the free option?

You



Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Keep **Hobby**—you don't need the $25 Pro plan.

Your screenshot shows **"$0/mo plus compute costs."** Hobby is the workspace plan; running the app and storing its database are billed separately.

For our current setup, the payment method is for the app's compute and persistent storage. You can return to **Billing** to add it without upgrading to Pro.

You

done

Worked for 23s

ChatGPT

The deployment plan is ready:

Resource	Monthly cost
Web service: 512 MB RAM, 0.5 CPU	$7.00
Persistent storage: 1 GB	$0.25
Estimated total	**$7.25**

That excludes taxes and usage overages. Your Hobby workspace stays $0/month. render.com

Saved accounts, assignments, and feedback will persist. Automatic deployments are off.

Approve deploying this plan at approximately $7.25/month? You'll also need to set the initial administrator email and password securely in Render.

You

do it

Take your curiosity further
Scan to get the ChatGPT app

Worked for 11s >

ChatGPT

The plan is approved, but deployment still needs your administrator credentials.

Open the Render deployment page and enter:

- `MENTOR_BOOTSTRAP_EMAIL` : your administrator login email.
- `MENTOR_BOOTSTRAP_PASSWORD` : a unique password of at least 12 characters.

Then click **Apply**. If those fields appear only afterward, set them under the service's **Environment** page; the initial deployment may fail until they're saved.

Keep the password out of chat. Tell me when saved, and I'll check the deployment.

You



ChatGPT

That screen shows **"No syncs yet"** and has no credentials form. Use Render's setup page instead:

1. Open Create Blueprint from your repository.
2. Select **mentor-matching** and branch **main**.
3. Enter your `MENTOR_BOOTSTRAP_EMAIL` and `MENTOR_BOOTSTRAP_PASSWORD` when prompted. Use at least 12 characters for the password.
4. Confirm the service and 1 GB disk, then click **Deploy Blueprint**.

Send me the resulting screen, keeping the password hidden.

You

okay, I also need this chat log- where can i get that so i can save it and push into github

Take your curiosity further
Scan to get the ChatGPT app

Worked for 22s

ChatGPT

For GitHub, save this conversation as a **Markdown file**:

1. Copy the conversation text from this chat.
2. In VS Code, create `docs/chat-logs/development-chat.md` .
3. Paste the conversation and save.
4. Review it for passwords or other private information before committing.
5. Run these commands in your project terminal:

```
git add docs/chat-logs/development-chat.md
git commit -m "Document AI-assisted development conversation"
git push
```

Keep your original prompts unchanged if this is part of the class's AI usage documentation. A development summary is useful too, but should be labeled separately from the actual transcript.

You



Worked for 13s

ChatGPT

The Docker build passed, but startup failed because the administrator email or password

You



Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

The build succeeded, but startup failed because the administrator credentials are missing or the password is too short.

1. Open your Render service.
2. Select **Environment → Edit** and set:

Key	Value
`MENTOR_BOOTSTRAP_EMAIL`	Your administrator email
`MENTOR_BOOTSTRAP_PASSWORD`	A unique password of at least 12 characters

3. Click **Save, rebuild, and deploy**.

Keep the password hidden. Tell me once saved, and I'll verify the deployment.

Take your curiosity further
Scan to get the ChatGPT app